ECE_3TC32_TP Compilation

AU 26-27 S1 — github/omgeta

1. Introduction

Compilateur est un programme qui traduit un "programme" d'un langage à un autre en préservant la sémantique (correct, rapide, optimisant).

Grandes étapes :

i. Analyse lexicale/Lexing :
texte → tokens via expressions régulières
ii. Analyse syntaxique/Parsing :
tokens → AST via une grammaire
iii. Analyse sémantique : variables/fonctions déclarées, types corrects, annotations de l'AST
iv. Optimisation
v. Géneration de code :
assembleur → object → binaire

Representation de base-R : $m = \sum_{i=0}^{n-1} b_i \cdot R^i$

Complément à deux :

i. $b_{N-1} \dots b_0 = -2^{N-1} b_{N-1} + \sum_{i<N-1} b_i 2^i$
ii. Opposé de x : $-x = 2^N - x$
iii. Intervalle : -2^{N-1} à $2^{N-1} - 1$
iv. Négation : $-x = \sim x + 1$
v. Débordement : tronquer (calculs dans $\mathbb{Z}/2^N\mathbb{Z}$)
vi. `+ - *` identiques signé/non signé

Boutisme :

i. Grand-boutien : écriture «naturelle»
ii. Petit-boutien : octet de poids faible à la basse ; $m = o_1 o_2 \Rightarrow m = o_1 + 256\, o_2$

Unité	Taille	C	courant
octet/byte (b)	8 bits	`char`	8
mot/word (w)	2 o	`short`	16
double (l)	4 o	`int`	32
quad (q)	8 o	`long`, `long long`, ptr	64

2. x86-64

Modèle :

i. Registres : rapides, peu nombreux
ii. RAM : grand tableau
iii. `%rip` : adresse de l'instruction courante
iv. Instruction lit/écrit registres ou RAM, ou `syscall`

Syntaxe AT&T : `op source, destination`

i. Registres préfixes par `%` (e.g. `$r9`)
ii. Constants préfixes par `$` (e.g. `$0x2A`) ; que les 32 bits signés
iii. Suffixe de taille : `b`=1, `w`=2, `l`=4, `q`=8 octets ; obligatoire si ambigu (e.g. `movq $42, (%rsp)`)

Registres Généraux

64	32	16	8	Rôle historique
`rax`	`eax`	`ax`	`al` (`ah`)	Accumulator / retour
`rbx`	`ebx`	`bx`	`bl` (`bh`)	Base / callee-saved
`rcx`	`ecx`	`cx`	`cl` (`ch`)	Counter / arg 4
`rdx`	`edx`	`dx`	`dl` (`dh`)	Data / arg 3
`rsi`	`esi`	`si`	`sil`	Source idx / arg 2
`rdi`	`edi`	`di`	`dil`	Dest. idx / arg 1
`rsp`	`esp`	`sp`	`spl`	Stack Pointer
`rbp`	`ebp`	`bp`	`bpl`	Base Pointer (frame)
`r8`–`r15`	`r8d`	`r8w`	`r8b`	libres (r8,r9 : args 5,6)

Addressage memoire :

i. Schema : `offset(base, index, scale)` = offset + base + index×scale
ii. offset : constante 32 bits signée ; scale ∈ {1, 2, 4, 8}
iii. `mov x(%rip), %rsi` : contenu au label `x`
iv. `lea x(%rip), %rsi` : adresse de `x`

Mémoire virtuelle : tableau de 2^{64} octets (48 bits utilisés), traduit par pages par l'OS, avec droits R/W/X.

Sections et données

```
.text       # code (executable)
.data       # donnees initialisees
.bss        # donnees init 0
.rodata     # lecture seule
.globl main  # exporte le symbole l'ELF
w:  .quad 0x1234567890  # 8o
x:  .long 12            # 4o
y:  .word 12            # 2o
z:  .byte 12          # 1o
s:  .ascii "Hello "  # 6o
t:  .asciz "World"   # 6o, avec \0
```

Instructions

Copie :

i. `mov[bwlq] s, d` : `d = s`
ii. `movz[bw][wlq] s, d` : extension par zéros
iii. `movs[bw|bl|bq|wl|wq|lq] s, d` : extension signée
iv. `cqto` : étend le signe de `%rax` dans `%rdx:%rax` (avant `idivq`)
v. `push s` / `pop d` : modifier pile

Arithmétique :

i. `add s, d` : `d += s` / `sub s, d` : `d -= s`
ii. `inc d` / `dec d` / `neg d`
iii. `imul s, d` : `d *= s` (mod 2^{64})
iv. `imul s` : `%rdx:%rax = %rax * s` ; `mul s` non signé
v. `idiv s` : divise `%rdx:%rax` par `s` ; `div s` non signé ; quotient → `%rax`, reste → `%rdx`.
vi. `lea s, d` : `d = &s`

Bit-à-bit :

i. `not d` / `and s, d` / `or s, d` / `xor s, d`
ii. `sal k, d` : `d <<= k` / `sar k, d` : `d >>= k` signé
iii. `shr k, d` : `d >>= k` logique

Comparaisons (flags uniquement) :

i. `cmp s, d` : `d - s`
ii. `test s, d` : `d & s`

Flags :

i. **CF** retenue / **ZF** nul / **SF** négatif / **OF** débordement signé / **PF** parité
ii. Modifiés par arithmétique, bit-à-bit, `cmp`, `test`
iii. Pas modifiés par `mov`, `lea`, `push`/`pop`, `jCC`, `setCC`

Sauts :

i. `jmp label` / `jmp *%rax` : inconditionnel
ii. `jCC label` / `cmovCC s, d` : si CC
iii. `setCC d` : `d = CC ? 1 : 0` (`d` 8 bits, ex. `%al`)

Conditions

CC	Sens	Flags
`e` / `z`	$d = s$ / nul	ZF
`ne` / `nz`	$d \neq s$	¬ZF
`s` / `ns`	négatif / positif	SF
`p` / `np`	parité paire / impaire	PF
`g` / `nle`	$d > s$ signé	¬(SF⊕OF)∧¬ZF
`ge` / `nl`	$d \geq s$ signé	¬(SF⊕OF)
`l` / `nge`	$d < s$ signé	SF⊕OF
`le` / `ng`	$d \leq s$ signé	(SF⊕OF)∨ZF
`a` / `nbe`	$d > s$ non signé	¬CF∧¬ZF
`ae` / `nb`	$d \geq s$ non signé	¬CF
`b` / `nae`	$d < s$ non signé	CF
`be` / `na`	$d \leq s$ non signé	CF∨ZF

`syscall` (Linux)

`%rax`	Nom	`%rdi`	`%rsi`	`%rdx`
0	read	fd	`char*`	count
1	write	fd	`char*`	count
12	brk	brk		
60	exit	code		

avec file descriptors : 0 = stdin, 1 = stdout, 2 = stderr.

3. Mémoire et fonctions

Memoire d'un programme :

i. Statique : `.text`/`.data`/`.bss`/`.rodata`, chargée depuis l'ELF
ii. Tas (heap) : à la demande (`malloc`), croît ↑
iii. Pile (stack) : fournie par l'OS, en haut, croît ↓

Pile :

i. `push s` : `sub $8,%rsp` + `mov s,(%rsp)`
ii. `pop d` : `mov (%rsp),d` + `add $8,%rsp`
iii. `call f` : `push` adresse retour + `jmp f`
iv. `ret` : `pop %rip`
v. `enter $N` : `push %rbp` + `mov %rsp,%rbp` + `sub $N,%rsp`
vi. `leave` : `mov %rbp,%rsp` + `pop %rbp`

Pile (frame)	Adresse
args 7+ de l'appelant	`16(%rbp)`, `24(%rbp)`
adresse de retour	`8(%rbp)`
ancien `%rbp`	← `%rbp`
variables locales	`-8(%rbp)`, `-16(%rbp)`
temporaires	← `%rsp` (sommet)

Convention d'appel (System V)

i. Arguments : `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`, puis pile
ii. Retour : `%rax`
iii. Callee-saved : `%rbx`, `%rbp`, `%r12`–`%r15`
iv. Caller-saved : `%rax`, `%rcx`, `%rdx`, `%rsi`, `%rdi`, `%r8`–`%r11`
v. Variadique (`printf`, `scanf`) : `%al` = 0
vi. Alignement : `%rsp` multiple de 16 au `call` (≡ 8 mod 16 à l'entrée, aligné après `push %rbp`)

Squelette :

```
f:  push %rbp            # frame appelant
    mov  %rsp, %rbp      # nouvelle frame
    sub  $16, %rsp       # locales (x16)
    mov  %rdi, -8(%rbp)  # arg 1
    ...                  # result -> %rax
    leave
    ret
```

libC

```
    .data
fmt_d:  .string "%d\n"
fmt_in: .string "%d"
    .text
    .globl main
main:
    push %rbp
    mov  %rsp, %rbp
    sub  $16, %rsp

    # scanf("%d", &n)
    lea  -8(%rbp), %rsi
    lea  fmt_in(%rip), %rdi
    xor  %eax, %eax
    call scanf
    movslq -8(%rbp), %rsi    # int -> long

    # printf("%d\n", n)
    lea  fmt_d(%rip), %rdi
    xor  %eax, %eax
    call printf
    xor  %eax, %eax          # return 0
    leave
    ret
```

4. Rappels de C

C est bas niveau, impératif, à passage par valeur et statiquement typé.

Types :

i. Fixent la taille (`sizeof(T)`) et le comportement
ii. Modificateurs : `signed`, `unsigned`, `const`
iii. Constantes : `42`, `0xA2`, `'H'`, `42LL`

Stockage :

i. Vie courte : registre
ii. Globale :
`.data` (init.), `.bss` (non init.), `.rodata` (`const`)
iii. Locale : pile (une copie par appel)

Valeur gauche : a une adresse, peut être à gauche de `=`

i. E.g. : `v`, `*v`, `*(v+2)`, `*(f(v))`
ii. Non : `42`, `v+42`, `f(v)`

Pointeurs :

i. `T* p` : pointeur vers un `T`
ii. `&a` : adresse de la l-value `a`
iii. `*p` : objet pointé

Arithmétique de pointeurs (`T* p, p1, p2`) :

i. `p + i` $= p + i \times$ `sizeof(T)`
ii. `p1 - p2` $= (p_1 - p_2)$ / `sizeof(T)`

Tableaux :

i. `t[i]` = `*(t+i)` ; `&t[0] == t`
ii. Statique : `int t[10];`
iii. Dynamique : `int* t = malloc(10*sizeof(int));`
iv. Linéarisé : `int t[99][88][77]`
$\Rightarrow$ `t[i][j][k]` = `*(t + i*88*77 + j*77 + k)`

Chaînes : tableau de `char` terminé par `'\0'` ; `"abc"` est un `const char*` en `.rodata`.

libC :

i. `malloc(n)` / `free(p)`
ii. `printf(fmt, ...)` / `scanf(fmt, ...)`

5. Grammaires

Grammaire formelle G :

i. Terminaux Σ (minuscules) : lettres des mots
ii. Non-terminaux N (majuscules) : variables
iii. Symbole de départ S
iv. Règles $(\Sigma \cup N)^+ \to (\Sigma \cup N)^*$

Langage $L(G)$: mots de Σ^* obtenus depuis S en appliquant les règles.

Notations :

i. $X \to \alpha$: règle ; α est une production de X
ii. $\alpha \Rightarrow \beta$: une étape (un NT remplacé)
iii. $\alpha \Rightarrow^* \beta$: plusieurs étapes (dérivation)
iv. $\alpha \Rightarrow_L \beta$: toujours le NT le plus à gauche

Décider $w \in L(G)$:

i. Grammaire quelconque : **indécidable**
ii. CFG (règles $N \to (\Sigma \cup N)^*$) : $O(n^3|G|)$, trop lent
iii. LL(k), LR(k) : **linéaire** $\Rightarrow$ utilisées en pratique

Inclusions :

i. LL(0) $\subset$ LL(1) $\subset$ LL(k) $\subset$ LR(k) $\subset$ CFG non ambiguës
ii. LR(0) $\subset$ SLR(1) $\subset$ LALR(1) $\subset$ LR(1)

Arbre de dérivation

i. Racine : S
Nœuds internes : NT
Feuilles : terminaux
ii. Enfants de X : $s_1 \dots s_k$ pour une règle $X \to s_1 \dots s_k$
iii. Mot de l'arbre : feuilles lues de gauche à droite
iv. $w \in L(G) \iff w$ est le mot d'un arbre

Ambiguë : un mot a plusieurs arbres (le sens du programme n'est plus unique).

Lever l'ambiguïté

Dangling else : rattacher `else` au `if` le plus imbriqué (IE = `if` avec son `else`) :

$$S \to \text{if } S \mid IE$$
$$IE \to \text{printf} \mid \text{if } IE \text{ else } S$$

Précédence : un NT par niveau, du plus faible au plus fort :

$$S_0 \to S_0 + S_1 \mid S_1$$
$$S_1 \to S_1 * S_2 \mid S_2$$
$$S_2 \to S_3\hat{}S_2 \mid S_3$$
$$S_3 \to \text{int} \mid (S_0)$$

Le sens de la récursion fixe l'associativité :

i. Récursion gauche ($S_0 \to S_0 + S_1$) : gauche, $1 - 2 - 3 = (1 - 2) - 3$
ii. Récursion droite ($S_2 \to S_3\hat{}S_2$) : droite, $2\hat{}3\hat{}4 = 2^{(3^4)}$

Algorithme général (CFG)

i. Binariser : $X \to s_1 \dots s_k$ devient $X \to s_1 F$, $F \to s_2 \dots s_k$ (F frais)
ii. $(i, j, \alpha) \in match$ ssi $\alpha \Rightarrow^* w_i \dots w_{j-1}$
iii. $w \in L(G) \iff (0, n, S) \in match$

match : plus petit ensemble contenant :

i. (i, i, α) si $\alpha \to \epsilon$; $(i, i+1, w_i)$
ii. (i, j, α) si $\alpha \to \beta$ et (i, j, β)
iii. (i, j, α) si $\alpha \to \beta\gamma$, (i, k, β) et (k, j, γ)

6. LL(1)

Analyse descendante : lecture **L**eft-to-right, dérivation **L**eftmost, k symboles d'avance.
LL(1) : **le prochain symbole suffit à choisir la règle**.

Null, Premier, Suivant

Calculés par point fixe (répéter jusqu'à stabilité).

Null : NT X avec $X \Rightarrow^* \epsilon$

i. $X \to \epsilon \Rightarrow X \in \text{Null}$
ii. $X \to Y_1 \dots Y_k$, tous $Y_i \in \text{Null} \Rightarrow X \in \text{Null}$
iii. Un terminal n'est jamais nul

Premier(α) : terminaux qui commencent un mot dérivé de α

i. $\text{Premier}(t) = \{t\}$ pour t terminal
ii. $\text{Premier}(X) = \bigcup_{X \to \alpha} \text{Premier}(\alpha)$
iii. $\text{Premier}(\alpha_1\alpha_2\dots) = \text{Premier}(\alpha_1)$, $\cup$ $\text{Premier}(\alpha_2\dots)$ si $\alpha_1 \in \text{Null}$

Suivant(X) : terminaux qui suivent X ($S \Rightarrow^* \alpha X t \beta$)

i. $\$ \in \text{Suivant}(S)$ (fin de l'entrée)
ii. Pour chaque $X \to \alpha Y \beta$: $\text{Premier}(\beta) \subseteq \text{Suivant}(Y)$
iii. Si de plus $\text{Null}(\beta)$: $\text{Suivant}(X) \subseteq \text{Suivant}(Y)$

Tester LL(1)

i. Calculer Null, puis Premier, puis Suivant
ii. Pour chaque règle $X \to \alpha$, la mettre dans $M[X, t]$:
   - pour tout $t \in \text{Premier}(\alpha)$
   - pour tout $t \in \text{Suivant}(X)$ si $\text{Null}(\alpha)$
iii. LL(1) $\iff$ au plus une règle par case

Conflits pour $X \to \alpha \mid \beta$:

i. $\text{Premier}(\alpha) \cap \text{Premier}(\beta) \neq \emptyset$
ii. α et β dérivent toutes deux ϵ
iii. $\alpha \Rightarrow^* \epsilon$ et $\text{Premier}(\beta) \cap \text{Suivant}(X) \neq \emptyset$

Rendre LL(1)

Problèmes :

i. Récursion gauche : $S \to S + T$ (boucle infinie)
ii. Préfixe commun : $S \to P \mid P + S$ (même Premier)

Solution : factoriser avec un NT de suite F_i.
$S_0 \to S_1 + S_0 \mid S_1$, $S_1 \to S_2 * S_1 \mid S_2$, $S_2 \to \text{int}$ devient :

$$S_0 \to S_1 F_0 \qquad F_0 \to +S_0 \mid \epsilon$$
$$S_1 \to S_2 F_1 \qquad F_1 \to *S_1 \mid \epsilon$$
$$S_2 \to \text{int}$$

i. $\text{Null} = \{F_0, F_1\}$
ii. $\text{Premier}(S_i) = \{\text{int}\}$, $\text{Premier}(F_0) = \{+\}$, $\text{Premier}(F_1) = \{*\}$
iii. $\text{Suivant}(S_0) = \text{Suivant}(F_0) = \{\$\}$
iv. $\text{Suivant}(S_1) = \text{Suivant}(F_1) = \{+, \$\}$
v. $\text{Suivant}(S_2) = \{*, +, \$\}$

Descente récursive

i. Une fonction par NT, qui renvoie l'arbre
ii. `peek()` : regarde le prochain symbole
iii. `eat()` : le consomme
iv. Règle choisie selon `peek()` (table LL(1))

```
def s0():              # S0 -> S1 + S0 | S1
    a = s1()
    if peek() != '+': return a
    eat(); return (a, '+', s0())
def s1():              # S1 -> S2 * S1 | S2
    a = s2()
    if peek() != '*': return a
    eat(); return (a, '*', s1())
def s2():              # S2 -> int
    a = peek(); assert a == 'int'
    eat(); return a
```

Attention : associativité droite ($1 - 2 - 3 = 1 - (2 - 3)$).
Pour gauche, une boucle :
`while peek()=='-': eat(); a=(a,'-',s1())`

7. Lexeur

Rôle : découper le texte en tokens, décrits par des expressions régulières.
e.g. `let x = 42` → LET VAR(x) EQ INT(42)

Langages réguliers : CFG avec règles $N \to \Sigma N \mid \epsilon$ (= regex = automates).

Expressions régulières

r	**Sens**	$L(r)$
$\emptyset$	vide	$\emptyset$
ϵ	mot vide	$\{\epsilon\}$
c	caractère	$\{c\}$
$[c_1 - c_2]$	classe	$\{c \mid c_1 \leq c \leq c_2\}$
$r_a \mid r_b$	union	$L(r_a) \cup L(r_b)$
$r_a r_b$	concaténation	$\{w_a w_b\}$
r^*	≥ 0 fois	$\{w_1 \dots w_k \mid k \geq 0\}$
r^+	≥ 1 fois	$\{w_1 \dots w_k \mid k \geq 1\}$

Algorithme du lexeur

Entrée : liste ordonnée (regex e_i, action a_i). Sur w :

i. w vide : fin
ii. Chercher le **plus long préfixe** reconnu par un e_i (aucun : erreur)
iii. Égalité : **plus petit** i (`let` avant `[a-z]+`)
iv. Exécuter a_i sur le préfixe, recommencer sur le reste

Lexeur séparé : décompose le problème, grammaires plus simples.

Reconnaître une regex

$match(R, i) = \{j \mid w_i \dots w_{j-1} \in L(R)\}$:

i. $match(\epsilon, i) = \{i\}$
ii. $match(c, i) = \{i+1\}$ si $w_i = c$
iii. $match([c_1, c_2], i) = \{i+1\}$ si $c_1 \leq w_i \leq c_2$
iv. $match(R_a|R_b, i) = match(R_a, i) \cup match(R_b, i)$
v. $match(R_a R_b, i) = \bigcup_{j \in match(R_a, i)} match(R_b, j)$
vi. $match(R^*, i)$: point fixe
$X = \{i\} \cup \bigcup_{j \in X} match(R, j)$

Automate fini déterministe :

i. États Q, finaux $F \subseteq Q$, initial i
ii. Transition $\delta : Q \times \Sigma \to Q$
iii. w accepté si $q_n \in F$, avec $q_0 = i$, $q_{k+1} = \delta(q_k, w_{k+1})$

En pratique : écrire des regex (lisibles), les compiler en automates (efficaces).

ocamllex

```
rule token = parse
| ['0'-'9']+ as s
          { `INT (int_of_string s) }
| "let"             { `LET }
| '+'               { `PLUS }
| [' ' '\t' '\n']+ { token lexbuf }
| ['a'-'z']['a'-'z' '0'-'9']* as id
                    { `VAR id }
| eof               { `EOF }
```

8. LR

Analyse ascendante : lecture **L**eft-to-right, dérivation **R**ightmost construite à l'envers, k symboles d'avance. Plus puissant que LL (récursion gauche OK).

Shift-reduce

Une pile de symboles + l'entrée. À chaque étape :

i. Shift : empiler le prochain symbole
ii. Reduce $A \to \alpha$: dépiler α, empiler A
iii. Accepter / refuser

L'action est lue dans une **table LR** : (état de la pile, prochain symbole) → action.

Analyser avec la table

i. Pile d'états (débute à 0) + pile de symboles
ii. Action[état, symbole] : shift s, reduce, accept ou erreur
iii. Après reduce $X \to \alpha$: dépiler $|\alpha|$, empiler Goto[état, X]
iv. Arbre : chaque reduce crée le nœud X avec les $|\alpha|$ fils dépilés

```
states <- [0] ; symbols <- []
loop:
  match action(states.top, input.peek):
  | Shift(s) ->
      symbols.push(input.pop())
      states.push(s)
  | Reduce(X->a) ->
      pop |a| de states et symbols
      symbols.push(X)
      states.push(goto(states.top, X))
  | Accept -> return OK
  | vide   -> erreur de syntaxe
```

Construire la table LR(1)

Item LR(1) $[X \to \alpha \bullet \beta,\ c]$: α déjà reconnu, β cherché, le tout suivi de c (LR(0) : sans c).

Clôture : pour $[X \to \alpha \bullet Y\beta, c] \in I$, ajouter $[Y \to \bullet\gamma, d]$ pour toute règle $Y \to \gamma$ et tout $d \in \text{Premier}(\beta c)$.

Transition : $\delta(I, X) =$ Clôture des $[Y \to \alpha X \bullet \gamma, c]$ pour $[Y \to \alpha \bullet X\gamma, c] \in I$.

Méthode :

i. Augmenter : $G \to S\$$;
$I_0 =$ Clôture($\{[G \to \bullet S\$, \$]\}$)
ii. $\forall I$, $\forall X$ après un $\bullet$: $J = \delta(I, X)$, numéroter si nouveau
iii. $[A \to \alpha \bullet a\beta, c]$: Action$[I, a]$ = shift $\delta(I, a)$
iv. $[A \to \alpha\bullet, c]$: Action$[I, c]$ = reduce $A \to \alpha$
v. $[G \to S\bullet, \$]$: Action$[I, \$]$ = accept
vi. NT X : Goto$[I, X] = \delta(I, X)$

Conflits (deux actions dans une case ⇒ pas LR(1)) :

i. Shift/reduce : e.g. $S \to S + S \mid \text{int}$, sur $+$
ii. Reduce/reduce

Exemple LR(1)

$G \to S\$$, $S \to CC$ (r_α), $C \to cC$ (r_β), $C \to d$ (r_γ)

i. $I_0 = \{[G \to \bullet S\$, \$], [S \to \bullet CC, \$],$
$[C \to \bullet cC, c/d], [C \to \bullet d, c/d]\}$
ii. $I_2 = \delta(I_0, C) = \{[S \to C \bullet C, \$], [C \to \bullet cC, \$],$
$[C \to \bullet d, \$]\}$
iii. $I_3 = \delta(I_0, c) = \{[C \to c \bullet C, c/d], [C \to \bullet cC, c/d],$
$[C \to \bullet d, c/d]\}$

	c	d	\$	S	C		c	d	\$	C
0	s3	s4		1	2	5			r_α	
1			acc			6	s6	s7		9
2	s6	s7			5	7			r_γ	
3	s3	s4			8	8	r_β	r_β		
4	r_γ	r_γ				9			r_β	

Trace `cdd` : 0 $\xrightarrow{s}$ 03 $\xrightarrow{s}$ 034 $\xrightarrow{r_\gamma}$ 038 $\xrightarrow{r_\beta}$ 02 $\xrightarrow{s}$ 027 $\xrightarrow{r_\gamma}$ 025 $\xrightarrow{r_\alpha}$ 01 acc

9. Logique combinatoire

Fonction logique $f(\vec{i}) = \vec{o}$; porte : son implémentation

a	b	AND	OR	XOR	NAND	$\neg a$
0	0	0	0	0	1	1
0	1	0	1	1	1	1
1	0	0	1	1	1	0
1	1	1	1	0	0	0

Tout avec NAND :

i. $\neg a = \text{NAND}(a, a)$
ii. $a \wedge b = \neg\text{NAND}(a, b)$
iii. $a \vee b = \text{NAND}(\neg a, \neg b)$
iv. $a \oplus b = \text{NAND}(\text{NAND}(a, n), \text{NAND}(b, n))$, avec $n = \text{NAND}(a, b)$
v. Mux $o = s\ ?\ i_1 : i_0$ $= \text{NAND}(\text{NAND}(\neg s, i_0), \text{NAND}(s, i_1))$

Additionneurs :

i. Half-adder : $s = a \oplus b$, $r = a \wedge b$
ii. Full-adder : $s = a \oplus b \oplus c$, $r = (a \wedge b) \vee (c \wedge (a \oplus b))$ $(= 1$ ssi $a + b + c \geq 2)$

Bus (k bits) :

i. Petit-boutien : $v(\vec{b}) = \sum_i b_i 2^i$; signe $= b_{k-1}$
ii. Addition (mod 2^k) : HA sur le bit 0, puis FA en chaîne ; mux de bus : un mux par bit

ALU (16 bits)

Tout est calculé, un arbre de mux sur $op_2 op_1 op_0$ choisit :

op	$\vec{z}$	**Réalisation**
0	$x + y$	additionneur
1	$\sim x$	NOT bit à bit
2	x & y	AND bit à bit
3	$x + 1$	retenue entrante 1
4	$x - y$	$x + \sim y + 1$
5	$x \oplus y$	XOR bit à bit
6	$x \mid y$	OR bit à bit
7	$x - 1$	x+`0xFFFF`

10. Logique séquentielle

Horloge ⇒ instants t ; état $\vec{u}_t$ $(\vec{u}_0 = \vec{0})$:
$(\vec{u}_{t+1}, \vec{o}_t) = f(\vec{u}_t, \vec{i}_t)$.
Pas une fonction mathématique ; les tables ne déterminent pas le circuit.

Composants :

i. Registre (delay) : $o_t = i_{t-1}$, $o_0 = 0$
ii. Mémoire 1 bit : $r_t = w_{t-1}\ ?\ v_{t-1} : r_{t-1}$ $\Rightarrow r = \text{DELAY}(\text{MUX}(w, r, v))$
iii. Registre 16 bits : 16 mémoires 1 bit, même w

Mémoire 2 lectures / 1 écriture

Lire $\vec{AL}_i$ (k bits) $\to \vec{VL}_i$ (16 bits) ; écrire $\vec{VE}$ en $\vec{AE}$ si e.

Récurrence sur k :

i. $k = 0$: un registre ; $\vec{VL}_1 = \vec{VL}_2 =$ contenu
ii. $k > 0$: m_0, m_1 sur $k - 1$ bits (sans le bit a)
- écrire m_b ssi $e \wedge (AE_a = b)$
- $\vec{VL}_i = \text{MUX}(AL_{i,a},\ m_0.\vec{VL}_i,\ m_1.\vec{VL}_i)$

RAM (init. à 0) / ROM (case i = tableau[i]) / Mixte (ROM programme + RAM données).

11. CPU

Composants :

i. Mémoire RAM/ROM : 2^{12} mots de 16 bits
ii. $2^4 = 16$ registres de 16 bits ; `PC` ; terminal 80×24

Instruction (16 bits) : `OP + 16*(R1 + 16*(R2 + 16*R3))`

i. Bits 0–3 : OP / 4–7 : R_1 (écrit)
ii. Bits 8–11 : R_2 / 12–15 : R_3 (lus)

OP	**Nom**	**Effet**
≥ 8	ALU	$R[R_1] = ALU(OP{-}8, R[R_2], R[R_3])$
7	CST HI	$R[R_1] = \overline{R_3 R_2} \times 2^8$
6	CST LOW	$R[R_1] = \overline{R_3 R_2}$
5	LOAD	$R[R_1] = RAM[R[R_2]]$
4	PC	$R[R_1] = PC$
3	OUTPUT	$Out(R[R_2], R[R_3])$
2	STORE	$RAM[R[R_2]] = R[R_3]$
1	JL	$PC = R[R_3]$ si $(R[R_2] < 0) \oplus R_1[0]$
0	JZ	$PC = R[R_3]$ si $(R[R_2] = 0) \oplus R_1[0]$

i. $\overline{R_3 R_2} = R_2 + 16 R_3$ (8 bits)
ii. $Out(x, y)$: caractère y en $(x\%80,\ x/80)$
iii. ALU : add 8, inv 9, et 10, inc 11, sub 12, xor 13, ou 14, dec 15
iv. $R_1[0] = 1$: saut inversé (non nul / non négatif)

Un cycle

i. Fetch : $ins = MEM[PC]$; decode OP, R_1, R_2, R_3
ii. Lire $A = R[R_2]$, $B = R[R_3]$ (toujours, sans risque)
iii. Calculer $ALU(OP_{0..2}, A, B)$ et $D = MEM[A]$
iv. Écrire $R[R_1]$ si $OP \geq 4$; $MEM[A] = B$ si $OP = 2$; sortie si $OP = 3$
v. $PC \leftarrow saut\ ?\ B : PC + 1$

Exemple (affiche A en boucle) :

```
ins_cst_low 0 0 ;      (* r0 = 0 *)
ins_cst_low (int_of_char 'A') 1 ;
ins_output 0 1 ;       (* Out(r0, r1) *)
ins_cst_low 2 2 ;      (* r2 = 2 *)
ins_jump_zero 0 2      (* PC = 2 *)
```